Microservices Assignment

Mohammed Zeeshan Praveen Meera

# AUTOMOTIVE ATTACK MANAGEMENT SYSTEM

Link to GitHub Repository for the application source code and code for Kubernetes deployment - https://github.com/Zeeshan8145/Automotive-Attack-Management-System and recording has been uploaded along with project folder

PA2577 Applied Cloud Computing and Big Data

# Section 1 Introduction

## 1.1 Application Overview

As part of my job responsibilities as an automotive cybersecurity engineer, I perform Threat Analysis and Risk Assessment (TARA) on my ECU in accordance with ISO 21434 and UN R155. One of the pain points of performing TARA is that there are multiple possible attacks, but not all of them may be probable. In order to reduce the speculative nature of attack step formation, the Automotive Attack Management System is something we are working on internally within my company to create our internal attack database for teams to use and help TARA authors search for existing internal attack records, retrieve external vulnerability information, and create internal attack records based on relevant external vulnerabilities. I adopted the existing system for my microservice assignment. The system now has distributed application components, each with a specific responsibility, that communicate with other components through REST APIs. Additionally, before adding a relevant external vulnerability to an internal attack record, it needs to be refined for automotive and organizational context. Internally within the company, we were going to do this via a form that the user has to fill in, but as part of the assignment and discussions with my colleagues, a background worker service now handles the processing and refinement of vulnerabilities asynchronously, while shared data stores are used for system records and job information.

As per the goals of the assignment, the project is being developed using Docker and deployed on Kubernetes, with the goal of making the system reproducible and portable, allowing other automotive cybersecurity teams or other users and enthusiasts to download the repository and tailor it accordingly.

Note: The entire project within my company and assignment has been inspired by https://www.autovia.app/

**General Abbreviations and Acronyms**

| Term | Definition |
|---|---|
| NVD | National Vulnerability Database |
| ECU | Electronic Control Unit |
| TARA | Threat Analysis and Risk Assessment |
| CVE | Common Vulnerabilities and Exposure |

## 1.2 System Architecture

The system has been delivered as a microservice architecture, where each component is separate from the others and communicates through REST APIs.

Technical details:

- Docker image for the components uses a lightweight Python base image.
- requirements.txt is used to list the dependencies for the app, pip is the Python command used to install them.
- Container port 8000 is configured for the frontend.

- Uvicorn was used to support and run the API as a server. Suggested through a Google search for more details: https://github.com/Kludex/uvicorn
- 0.0.0.0 tells Uvicorn to listen on all network interfaces inside the container, and app.main:app is used to load the module and run the application accordingly.

Dependency details:

- FastAPI – Python support for defining HTTP endpoints and handling requests.
- Uvicorn – Used to run the apps
- Jinja2 – Renders HTML templates. I have not worked on web development before and found it through searching: https://github.com/pallets/jinja (used for Frontend Dashboard)
- HTTPX – Used to send HTTP requests.
- python-multipart – Included since our use case requires parsing user submitted HTML form data (used for Frontend Dashboard)
- pymongo - to interact with the MongoDB database ( for all service which need to connect to the database)

The components are:

Note: The following section is pretty comprehensive on the end points, for a quick overview please have a look at the github page)

**a) Frontend Dashboard (Build for this assignment)** : It provides the browser-based interface where users can search for vulnerabilities, approve relevant external vulnerabilities for internal records, and monitor background job processing. It is the main user-facing interface where a user can search and view results.

End Points:

- GET / — Dashboard: Defines an HTTP GET endpoint for the browser to access the dashboard. It has default values to display, but it also asynchronously requests statistics from the Internal Attack Lookup service through an API. If a JSON response is received, it is parsed and the statistics are displayed. If the request fails, the default values are used.
- GET /search — Internal Attack Search with CVE Fallback: Defines an HTTP GET endpoint to search for an attack using keywords entered by the user.
  - If the query is not empty, the first try block searches the internal automotive attack database through the Internal Attack Lookup microservice's API endpoint /attacks/search. It expects a JSON response containing the search results.
  - If results are found, they are displayed. Otherwise, the code moves to the else block and sends the query to the CVE Lookup microservice through its API endpoint /cves/search. It again expects a JSON response containing the results.
- GET /search-cves — Independent CVE Search: Based on feedback from a friend, it was suggested that whenever a keyword is searched, the system should not only return matches from the internal attack database. It would also make sense to allow users to independently browse attacks from a wider database if they wish.
  - If the query is not empty, it expects a JSON response containing the search results from the CVE Lookup microservice.
- POST /approve-cve — Approve and Create an Internal Attack: When a user approves a CVE from the external database and wants to add it to the internal database, the system defines a POST endpoint called /approve-cve

- The submitted data is converted into JSON and sent to the Internal Attack Lookup service through its POST endpoint /attacks/from-cve.
- From this point onwards, the frontend receives a job ID because, as per the implementation, a background worker analyses the content and adds the resulting record through the internal processing workflow.
- The frontend polls the job status every three seconds. If the job fails, it displays an error message. When the job status becomes Completed, it retrieves the saved attack record from the Internal Attack Lookup service through the /attacks/search endpoint, which returns JSON. The frontend parses the response and displays the record on the dashboard.

o GET /health — Health Check: Defines a GET endpoint at /health to check whether the frontend service is responding. This is standard practice.

**b) Internal Attack Lookup Service (Adopted for this assignment) :** This service has been adopted from existing logic that we are building upon internally. This is the core of the system, which manages internal searches and internal attack record creation. It provides endpoints for searching internal attack records, retrieving relevant statistics, and initiating the creation of internal attack records from external databases. It queries the database for existing records and coordinates the submission of background jobs when a user requests a new internal attack record.

End Points:

o GET /stats for the Dashboard: Defines an HTTP GET endpoint /stats to send basic statistics for the frontend to display. It execute the stats function and counts records in the databased in known attacks, cve cache, and pending lookups. If successfully retrieved it returns them as a JSON to the frontend else throws a error

o POST /attacks — Internal Attack service to create attack directly:One of the use cases for the microservice is to create an attack. To do so, it defines an HTTP POST endpoint to create an attack using the BaseModel.
- First, it performs a sanity check on the model and, once done, inserts the attack into the MongoDB database and retrieves the database-generated ID.
- With the retrieved generated ID, it converts it into a string and returns the saved attack to the frontend as JSON to be displayed.

o POST /attacks/from-eve — Internal Attack service to schedule a job once a user wants to create an attack record based on external database: Another use cases for the microservice is when an user wants to create an attack based on external database, it defines an HTTP post endpoint to schedule a job to the background worker to perform the necessary context additions and insert the attack record
- First, it performs a sanity check on the model and, once done, checks the database for an existing record so that it does not queue a job for a CVE that has already been approved and added to the database.
- If it finds that the record already exists, it informs the user; otherwise, it continues to schedule a unique job ID (to retrieve the status later) with a status of Pending and fields such as job type, attempts, payload, and timestamps, and inserts them into the database's attack_job index. This helps ensure that if the CVE is currently being processed, the user can be alerted to the same.

o GET /jobs/{job_id} —Complementary Endpoint to Check and Retrieve the Status of a Scheduled Job: Most of the time, the job should be scheduled and forgotten about for

the background worker to do its work, but there is a use case where the user might want to know whether a queued job has finished

- To support this, we define a GET API endpoint that takes the job_id identifier, tries to find the job, and, if found, returns fields such as ID, status, attempts, attack ID, created_at, updated_at, and last_error, if available. Otherwise, if no jobs exist, it throws a Not Found error.

- GET /attacks/search — Internal Attack Service to Search for an Attack in the Database: The main use case is that the Internal Attack Service shall search the database for attacks and return the hits to be displayed in the frontend. To do so, we define a GET endpoint to search for attacks based on the query from a user.
  - First, we take the query and try to search by the attack ID, since it could be simpler. If we find a match, we return the attack based on the attack ID. If no ID matches, we search based on text fields. We then search the database using the query in three fields: title, description, and keywords. Once we find a hit, we return it as JSON for the frontend to display.
- GET /health — Health Check: Defines a GET endpoint at /health to check whether the service is healthy and is able to ping MongoDB, since without a database connection the microservice is rendered with no use

**c) CVE Lookup Service (Adopted for this assignment):** This service has been adopted from existing logic that we are building upon internally. This service processes external searches against external databases. It processes external search requests and prepares vulnerability information for presentation and user review. This functionality allows the application to extend its internal knowledge base using external vulnerability scores. Today, it communicates only with https://nvd.nist.gov/developers/vulnerabilities , which is the National Vulnerability Database maintained by NIST.

End Points:

- GET /CVE/SEARCH: Defines an HTTP GET endpoint for performing the search as described in the frontend section. It takes the query from the frontend user's input and validates the query.
  - Once the query is validated, it is first checked in the local MongoDB cache. If anything is found, it returns the cached JSON to the frontend; otherwise, it proceeds to send a request to the NVD API.
  - It sends an internal GET request with the query to check the NVD public database. The URL is configured at the beginning, as per https://nvd.nist.gov/developers/vulnerabilities , with a 15-second timeout. We receive a JSON response from the NVD database based on the matches for our query.
  - Then, we clean the data from the NVD JSON, with parameters such as CVE ID, description, English-based descriptions for simplicity, and mapped dates. Once this is done, we update these records in the cache and return JSON to the API client, in this case the frontend, for processing.
- GET /cve/{cve_id} : Also, in order to retrieve a CVE by ID, the service defines an HTTP GET endpoint to search for a CVE using its ID.
  - It passes the ID to the get_cve function, and if the CVE is found in the database cache, it returns the CVE as JSON with a 200 OK status.
- GET /health — Health Check: Defines a GET endpoint at /health to check whether the service is healthy and is able to ping MongoDB, since without a database connection the microservice is rendered with no use

**d) Threat Analyser Background Service (Built for this assignment):** This service has been developed to process threat analysis jobs in the background.
Working:

- First, it sets the required URL for the worker to interact with the database and configures the polling interval, maximum attempts, and lease seconds to ensure it knows how long the worker should sleep before checking the queue again when idle, how many times it can attempt a job before marking it as failed, and how long it can hold a job before the system can recover it if the worker goes down abruptly.
- We have used the stable database and collections, with jobs, attacks, and db configured to the known collections in the database.
- Then, we initialize the worker, which pings the database to establish a connection, sets up its indexes, and initializes the attack ID counter and the worker ID.
- Then, we have a function that is used to calculate the threat score before adding the attack ID. Currently, I have a pseudo-configuration as shown below to calculate the custom threat score.

```
Year rules:
    2015 through the current year: +5 points
    2005 through 2014:             +4 points
    2000 through 2004:             +3 points
    Before 2000:                   +2 points

Description rules:
    Whole phrase 'remote attack' or 'automotive attack': +3 points
    Whole phrase 'root access' or whole word 'privilege': +2 points
```

- Then, we have process_job(), which contains the main logic for a worker to process a claimed job. First, it retrieves the job's payload and job ID. It validates and normalizes the CVE ID, title, description, keywords, and applicable ECU values. It then calls the calculate_threat_score() function to calculate the threat score and obtain its breakdown.
- Next, it checks whether an internal attack record for the CVE already exists in the database to avoid duplicates. If an existing record is found, it reuses the existing internal Attack ID. Otherwise, it generates a new internal Attack ID, builds the attack document with the relevant information and threat score, and inserts it into MongoDB. It also handles a duplicate-key error in case another worker has already inserted a record for the same CVE.
- Once processing succeeds, the function updates the job's status to Completed, stores the internal Attack ID in the job document, and removes the worker ownership and lease information.
- If validation fails, the job is marked as Failed. If a MongoDB operation fails, the job is returned to Pending for another attempt unless the maximum number of attempts has been reached, in which case it is marked as Failed. The function also records error information and logs the failure

## 1.3 Cloud and Microservice Architecture Patterns:

**Microservice architecture patterns:**

a) Shared Database Server Pattern: As per [1], this pattern describes multiple services using a common database server. The Automotive Attack Management System follows this pattern since the Internal Attack Lookup and Background Threat Analyser share access to MongoDB for attack records and

background jobs. The CEV lookup also uses the data for cached CVE information when configured accordingly. This approach reduces infrastructure complexity and makes shared application data accessible to the services that need it. However, it introduces coupling through the shared database, and availability, performance, and changes to the shared database can affect multiple services. Currently, in the Kubernetes deployment, we deploy only one MongoDB instance through a Kubernetes StatefulSet with persistent storage. Persistent storage helps retain data across Pod replacements. A multi-node deployment could be introduced in the future to follow a database cluster pattern for greater availability.

b) Server-side Service Discovery Pattern: As per [1], this pattern addresses how a service locates another service without depending on a fixed address for each individual service instance. The Automotive Attack Management System uses Kubernetes Services and cluster DNS for service discovery. For example, the frontend uses service URLs such as http:internal-attack-lookup:8000 and http:cve-lookup:8000 . Kubernetes provides stable service addresses and routes requests to eligible Pod endpoints. This allows Pods to be replaced without requiring the frontend to maintain their individual IP addresses.

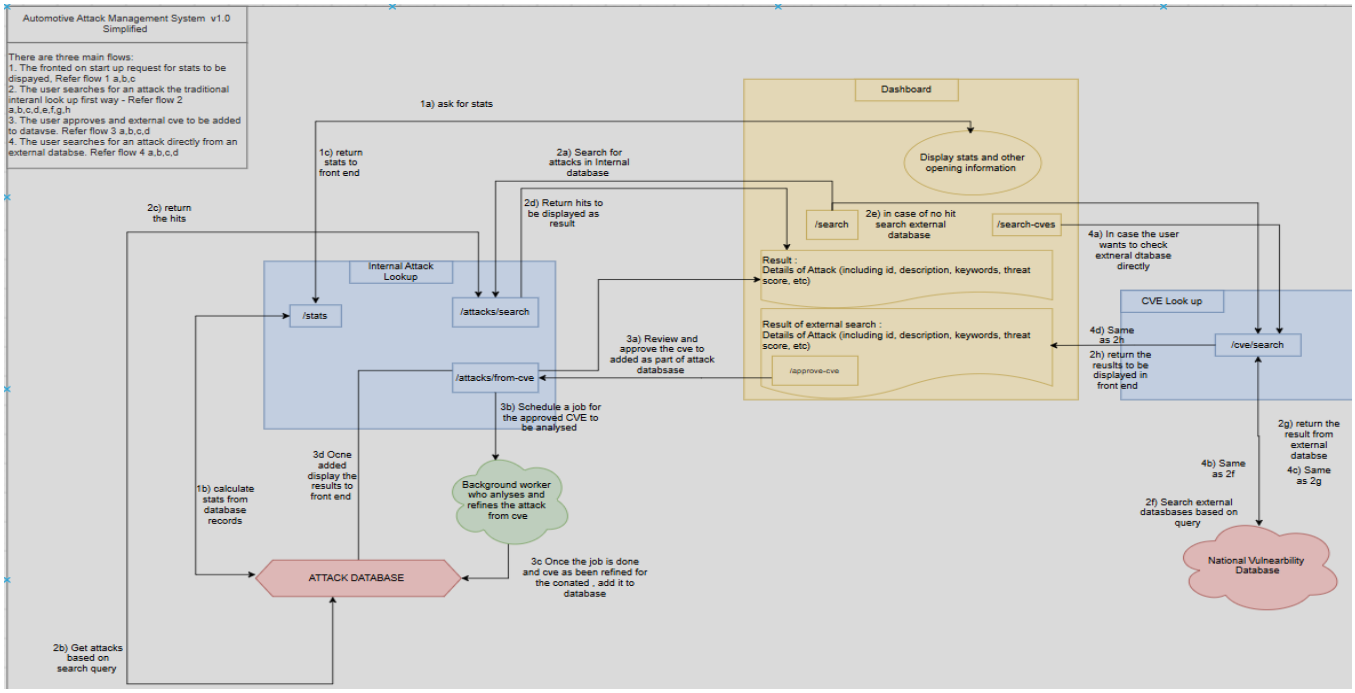
**Cloud architecture patterns:**

a)Horizontal Scaling Compute Pattern: The Automotive Attack Management System separates its services into independent deployment units. Kubernetes can run multiple replicas of eligible stateless services, allowing request-handling capacity to be increased without scaling every service at the same time. The frontend, Internal Attack Lookup service, and CVE Lookup service are candidates for horizontal scaling. The Background Threat Analyzer may also use multiple workers, provided that job claiming and processing are made safe for concurrent execution. This is in accordance with the Horizontal Scaling pattern discussed in [2].

b) Queue Centric Flow Pattern: In the Automotive Attack Management System, the Internal Attack Lookup service creates jobs in MongoDB, and the Background Threat Analyzer processes them independently. This approach supports asynchronous processing and prevents the API from having to perform all analysis during the initial request. This is in partial accordance with Queue Centric Flow Pattern discussed in [2].

c) Auto Scaling and Node Failure Pattern: These two cloud patterns discussed in [2] talks about extending the number of applications automatically according to workload and the ability to restart faked containers and pods. Fortunately, for the Automotive Attack Management System, these patterns can be realized through the Kubernetes deployment, Kubernetes can be extended to adjust the number of application replicas automatically according to workload or resource metrics and Kubernetes can also restart failed containers and replace failed Pods according to the relevant workload configuration.

## 1.4 Full Architecture Flow Diagram : For clearer view - [click here]




## 1.6 Docker Images and Kubernetes Deployment (I would strongly suggest to go through [Github page] for better understanding)

As required by the assignment, the Automotive Attack Management System uses Docker to package services into images and Kubernetes to orchestrate their deployment. A separate Dockerfile is provided for each of the four microservices, allowing each service to be built, deployed, run, and updated independently. Additionally, MongoDB uses the official MongoDB 8 container image. The deployment.yaml file contains the Kubernetes configuration required to deploy the application within a Kubernetes cluster. It includes:

- Deployments for the frontend, Internal Attack Lookup Service, CVE Lookup Service, and Background Threat Analyser.
- Services to provide network endpoints for the application components.
- A MongoDB StatefulSet to manage the database.
- Persistent storage to retain database data beyond the lifecycle of an individual pod.

The project has been developed and tested using Docker Desktop with a local Kubernetes environment. This allows the application to run without requiring a paid cloud-based Kubernetes service.

High-Level Deployment Steps

For detailed instructions, please refer to the README.md file in the project's GitHub repository.

- Clone the repository.

- Start a supported local Kubernetes cluster.
- Build the application images locally using Docker or pull the images from the appropriate registry.
- Make the images available to the Kubernetes environment.
- Apply the deployment.yaml configuration using the appropriate Kubernetes command.
- Verify the status of the pods, services, and persistent storage.
- Access the frontend through a web browser and test the complete end-to-end system workflow.

The most common issue I found was that sometimes it takes an unusually long time for mongodb to be ready, and without the database the service will fail as well (Kubernetes will try to keep restarting them). Usually just wait a bit if you feel there is an issue, run "kubectl get pods" and "kubectl logs mongodb-0" and try to check the logs for issues. If the pods are not starting despite mongodb running "kubectl logs <POD_NAME>" and trying to check , common issues would be image pull failure if local images are used, startup errors or just bad configs.

One thing to note is that, initially I had the "frontend" as "NodePort" kind service to expose URLs but for some reason i couldn't access the frontend mainly because of WSL2/DockerDesktop configurations making the site unreachable, as a quick fix I opened a separate terminal and ran "kubectl port-forward service/threat-frontend 8000:8000" which made the system accessible in "http://localhost:8000" . This issue appears to be related to my use of WSL2 for development and deployment, where host networking and Kubernetes networking can interact in unexpected ways. For users running the project directly from PowerShell on Windows, the NodePort configuration may work without additional changes, provided that Docker Desktop, Kubernetes, and the host networking configuration are set up correctly. However as a permanent fix, I changed the "frontend" as a "LoadBalancer" kind service which made it available "http://localhost:8000" so any user shouldn't see the issue.

# Section 2: Benefits, Challenges/Security Considerations, Proposed v2.0:

## 2.1 Benefits

The current microservice architecture provides multiple benefits. The most obvious is that all services have distinct responsibilities and can be deployed and scaled independently and separately, as long as their dependencies and resource requirements are addressed. Additionally, background analysis jobs can be processed independently of the initial user request. Kubernetes Services and DNS allow application components to communicate without depending on individual Pod IP addresses and also allow us to set up MongoDB persistent volumes to retain data when the database Pod is restarted or replaced.

## 2.2 Challenges and Security Considerations

Though the code could be refactored and the frontend could be made more presentable, after discussing with a colleague, an important concern was raised regarding a missing aspect of the project: since automated attack information may contain confidential or security-sensitive information, access to such records must be restricted to authenticated and authorized users.

Additionally, the choice to use a shared database increases the impact of incorrectly configured permissions. If every authorized company employee has unrestricted access to the same data, a compromised service or an unauthorized user could gain access to information beyond the intended scope, which also introduces a security problem.

Therefore, the main challenges are authentication, where users must establish their identities before accessing records, and data isolation, particularly in a multi-project environment where each project and its project members may require access to their own data and associated jobs. Only authorized project members should be able to access that project's records. Another security consideration is that when users access sensitive attack records, there should be logs for monitoring and accountability to prevent repudiation. These challenges cannot be fully addressed by the existing implementation and would require security to be incorporated into the API authentication, identity management, and database access design.

## 2.3 Proposed v2.0

To mitigate these challenges and security concerns, the following can be done.

The system should implement authentication and authorization before returning attack information or performing protected operations. The system should require users to sign in and assign permissions according to their roles and responsibilities. We should also establish dedicated identities for backend services and restrict each service to the minimum read and write permissions required. Authorization must be enforced by the backend rather than relying only on hiding dashboard controls. A user who cannot see an operation in the frontend must not be able to bypass that restriction by calling the API directly.

Additionally, to support scalability and project-level access requirements, the architecture should evolve towards explicit project-level data ownership and access boundaries. Possible approaches include separate databases for individual projects, carefully enforced project identifiers and authorization checks, or separate database deployments where stronger isolation is required .To strengthen security, Docker containers should follow security best practices, including running applications as non root users, avoiding hardcoded credentials in container images, and exposing only the necessary network ports. Kubernetes Secrets can be used to manage sensitive values, such as database credentials and service authentication keys. Access to these secrets should be restricted through role-based access control (RBAC), following the principle of least privilege. Each workload should use an appropriately restricted Kubernetes ServiceAccount, and NetworkPolicies could limit database connectivity to approved application workloads where supported by the cluster's networking implementation. It is important to note that application-level authentication, authorization, and data access controls are the primary security measures. Kubernetes RBAC, Secrets management, ServiceAccounts, and NetworkPolicies provide complementary layers of defense and should not replace application-level security control

In conclusion, in v2.0 as a starting point authentication and authorization could be introduced at the Internal Attack Lookup API to ensure that only authorized users or services can submit attack-creation requests. For example, when a user approves a CVE through the dashboard, the backend could verify the

user's identity and project-level permissions before submitting the job. The worker would then process the authorized job and store the resulting record. This would prevent unauthorized users from bypassing the dashboard and directly invoking the attack creation endpoint. Also a status check API for background workers would be nice.

## Section 4 References And Links:

[1] Taibi, Davide, Valentina Lenarduzzi, and Claus Pahl. "Architectural patterns for microservices: a systematic mapping study." CLOSER 2018: Proceedings of the 8th International Conference on Cloud Computing and Services Science; Funchal, Madeira, Portugal, 19-21 March 2018. SciTePress, 2018.
[2] Wilder, Bill. Cloud Architecture Patterns. O'Reilly Media, Inc., 2012
[3] Link to GitHub Repository for the application source code and code for Kubernetes deployment - https://github.com/Zeeshan8145/Automotive-Attack-Management-System
[4] Link to DockerHub for the images - https://hub.docker.com/repositories/zeeshan8145
[5] Recording: Uploaded along with project folder